

Internals Linux pour le debug

Internals Linux pour le debug

Les briques système que tout diagnostic présuppose

Objectifs du module

À la fin de ce module, vous serez capable de :

- Distinguer un processus d'un thread et lire leur état dans /proc
- Expliquer ce que fait l'ordonnanceur et ses principaux états
- Séparer clairement espace utilisateur et espace noyau
- Reconnaître un appel système et un signal dans une trace

Plan

1. Processus, threads et ordonnancement
2. User space et kernel space
3. Appels système et signaux
4. Démo et exercice
5. Récap et questions

1. Processus, threads et ordonnancement

Processus vs thread

“ Un processus est un contexte d'exécution isolé ; un thread partage l'espace mémoire de son processus. ”

Points essentiels :

- PID identifie un processus, TID identifie un thread au sein d'un processus
- Les threads d'un processus partagent mémoire, descripteurs de fichiers et table des signaux
- `/proc/<pid>/task/` liste tous les threads d'un processus

États d'un processus

État	Signification
R	Runnable, en cours ou prêt à tourner
S	Interruptible sleep, attend un événement
D	Uninterruptible sleep, bloqué en noyau (souvent I/O)
Z	Zombie, terminé mais non reaped
T	Stopped, suspendu (SIGSTOP)

Un processus en D persistant pointe presque toujours vers un I/O lent ou un driver.

Ordonnancement en une slide

- Le noyau alloue du temps CPU par quantum
- Plusieurs classes : CFS (défaut), realtime (SCHED_FIFO, SCHED_RR), deadline
- Priorité visible via `nice` et `priority` dans `ps`
- Context switch = sauvegarder l'état courant et restaurer un autre thread

Observer : `pidstat -w` montre les context switches par process.

2. User space et kernel space

Le mur user / kernel

User space

- Applications, bibliothèques
- Ne peut pas accéder au matériel
- Isolation via la MMU

Kernel space

- Ordonnanceur, drivers, FS, stack réseau
- Accès matériel direct
- Un seul espace partagé par tous

Passage obligatoire : l'appel système.

Code et commandes

```
# Voir les threads d'un processus
ls /proc/<pid>/task/

# Voir l'état courant
cat /proc/<pid>/status | grep -E 'State|Threads|VmRSS'

# Voir l'exécutable et la cmdline
readlink /proc/<pid>/exe
tr '\0' ' ' < /proc/<pid>/cmdline
```

3. Appels système et signaux

Appels système

“ Un syscall est la porte d'entrée contrôlée entre user space et kernel. ”

Catégories utiles au debug :

- Fichiers : `open`, `read`, `write`, `close`, `stat`
- Processus : `fork`, `execve`, `wait4`, `exit_group`
- Synchro : `futex`, `poll`, `epoll_wait`
- Réseau : `socket`, `connect`, `sendto`, `recvfrom`

Une trace qui bloque sur un syscall pointe l'origine du freeze.

Signaux à connaître

Signal	Effet par défaut	Usage debug
SIGTERM (15)	Termine proprement	Shutdown gracieux
SIGKILL (9)	Termine sans recours	Dernier recours
SIGSEGV (11)	Core dump	Bug mémoire
SIGHUP (1)	Rechargement config	Recharge sans kill
SIGSTOP / SIGCONT	Pause / reprise	Figer pour inspection

Exercice d'application

Exercice 1 : cartographier un processus suspect avec /proc et ps

- Durée : 20 min
- Fichier : `exercices/exercice-01.md`

